# UNIVERSIDAD NACIONAL DE JUJUY – FACULTAD DE INGENIERÍA

## Carrera: Analista Programador Universitario (APU)

Cátedra: Herramientas Informáticas Avanzadas

**Trabajo Práctico:** Diseño y Optimización de Bases de Datos
**Docente a Cargo:** Prof. Casasola
**Equipo de Desarrollo:**


- Antivilo, Aldo Dante — LU: 5381
- Caucota, Gonzalo — LU: 2423
- Gaspar, Facundo Ezequiel — LU: 5309


**Fecha de Presentación:** Jueves 15 de Octubre de 2026 | **Fecha de Defensa:** Viernes 16 de Octubre de 2026
**Repositorio Oficial de Código y Scripts:** `https://github.com/AldoDante/Tp_Optimizacion.git`
**Entorno de Laboratorio:** Contenedor Docker Oficial MySQL 8.0 Community Server (Motor Transaccional InnoDB)

---

# ÍNDICE GENERAL DEL INFORME


- PARTE I — ANÁLISIS DEL PROBLEMA Y DISEÑO DE DATOS (PUNTOS 1 AL 5)
  - Punto 1: Análisis del Dominio y Reglas de Negocio
  - Punto 2: Modelo Relacional y Transformaciones
  - Punto 3: Normalización Formal (1FN, 2FN y 3FN)
  - Punto 4: Integridad Física, Declarativa y Reglas de Negocio
  - Punto 5: Normalización versus Desnormalización Controlada
- PARTE II — DISEÑO FÍSICO Y ALMACENAMIENTO (PUNTOS 6 AL 10)
  - Punto 6: Implementación Física (Script DDL Definitivo)
  - Punto 7: Matriz de Selección Justificada de Tipos de Datos
  - Punto 8: Claves Físicas y Mecanismos de Integridad
  - Punto 9: Motor y Estructuras Transaccionales de Almacenamiento (InnoDB)
  - Punto 10: Estrategias de Almacenamiento, Páginas de Disco y Rendimiento
- PARTE III — ÍNDICES Y ESTRUCTURAS DE ACCESO (PUNTOS 11 AL 18)
  - Punto 11: Estrategia General de Índices del Sistema
  - Punto 12: Arquitectura Interna de Árboles B-Tree y B+Tree
  - Punto 13: Otros Tipos de Estructuras de Indexación
  - Punto 14: Índices Compuestos y Demostración de la Regla del Prefijo Izquierdo
  - Punto 15: Índices de Cobertura (Covering Indexes)
  - Punto 16: Índices Funcionales o Basados en Expresiones en MySQL 8.0
  - Punto 17: Análisis Matemático de Cardinalidad y Selectividad
  - Punto 18: ¿Por Qué No Indexar Todos los Campos? (Costo DML)
- PARTE IV — DATOS, CARGA MASIVA Y METODOLOGÍA EXPERIMENTAL (PUNTOS 19 AL 22)
  - Punto 19: Generación Reproducible de Datos de Prueba
  - Punto 20: Carga Masiva Crítica (8.100.000 de Registros)
  - Punto 21: Distribución Estadística de los Datos y Sesgos
  - Punto 22: Metodología Experimental de Benchmarking
- PARTE V — SQL Y CONSULTAS DEL SISTEMA (PUNTOS 23 AL 28)
  - Punto 23: Telemetría Experimental: Cold Cache versus Warm Cache
  - Punto 24: Consultas SQL de Dificultad Progresiva
  - Punto 25: Desarrollo Completo de las 15 Consultas del Sistema
  - Punto 26: Consultas Avanzadas sobre Disponibilidad y No Solapamiento
  - Punto 27: Consultas Históricas y Temporales
  - Punto 28: Transporte de Datos y Paginación Profunda (OFFSET vs Keyset)
- PARTE VI — OPTIMIZADOR, SARGABILIDAD Y PLANES DE EJECUCIÓN (PUNTOS 29 AL 33)
  - Punto 29: Diagnóstico con EXPLAIN y EXPLAIN ANALYZE
  - Punto 30: SARGabilidad y Predicados Optimizables
  - Punto 31: Estadísticas del Optimizador e Información de Cardinalidad
  - Punto 32: Modelo de Costos y Selección del Plan de Ejecución
  - Punto 33: Optimización de Algoritmos de JOIN
- PARTE VII — OPTIMIZACIÓN DE CONSULTAS Y TABLA COMPARATIVA (PUNTOS 34 AL 37)

    - Puntos 34 al 36: Metodología de Optimización Aplicada
    - Punto 37: La Gran Tabla Comparativa "Antes vs. Después" de las 8 Consultas Críticas
- PARTE VIII — TRANSACCIONES Y CONCURRENCIA (PUNTOS 38 AL 42)
    - Punto 38: Mecanismos de Bloqueo a Nivel de Tabla y Fila (S y X)
    - Punto 39: Concurrencia entre Dos o Más Conexiones Simultáneas
    - Punto 40: Niveles de Aislamiento ANSI SQL y Fenómenos Anómalos
    - Punto 41: Simulación Experimental de Deadlocks en 4 Pasos
    - Punto 42: Herramientas Forenses y Diagnóstico de Bloqueos en InnoDB
- PARTE IX — MEMORIA, SERVIDOR Y RENDIMIENTO (PUNTOS 43 AL 48)
    - Punto 43: Configuración del Motor y Archivo my.cnf
    - Punto 44: Telemetría Real del InnoDB Buffer Pool (Hit Ratio del 86,11%)
    - Punto 45: Identificación de Cuellos de Botella: CPU, Disco y Red
    - Punto 46: Evaluación de Pools de Conexión (Connection Pooling)
    - Punto 47: Configuración y Evidencia del Slow Query Log
    - Punto 48: Tareas Preventivas de Mantenimiento de Tablas e Índices
- PARTE X — ESCALABILIDAD, PARTICIONAMIENTO Y ESTRUCTURAS AVANZADAS (PUNTOS 49 AL 52)
    - Punto 49: Análisis Comparativo: Escalabilidad Vertical vs. Horizontal
    - Punto 50: Implementación Física de Particionamiento Horizontal por Rangos
    - Punto 51: Demostración Empírica de Partition Pruning en Planes de Ejecución
    - Punto 52: Vistas Estándar versus Vistas Materializadas
- CONCLUSIONES FINALES


---

# PARTE I — ANÁLISIS DEL PROBLEMA Y DISEÑO DE DATOS

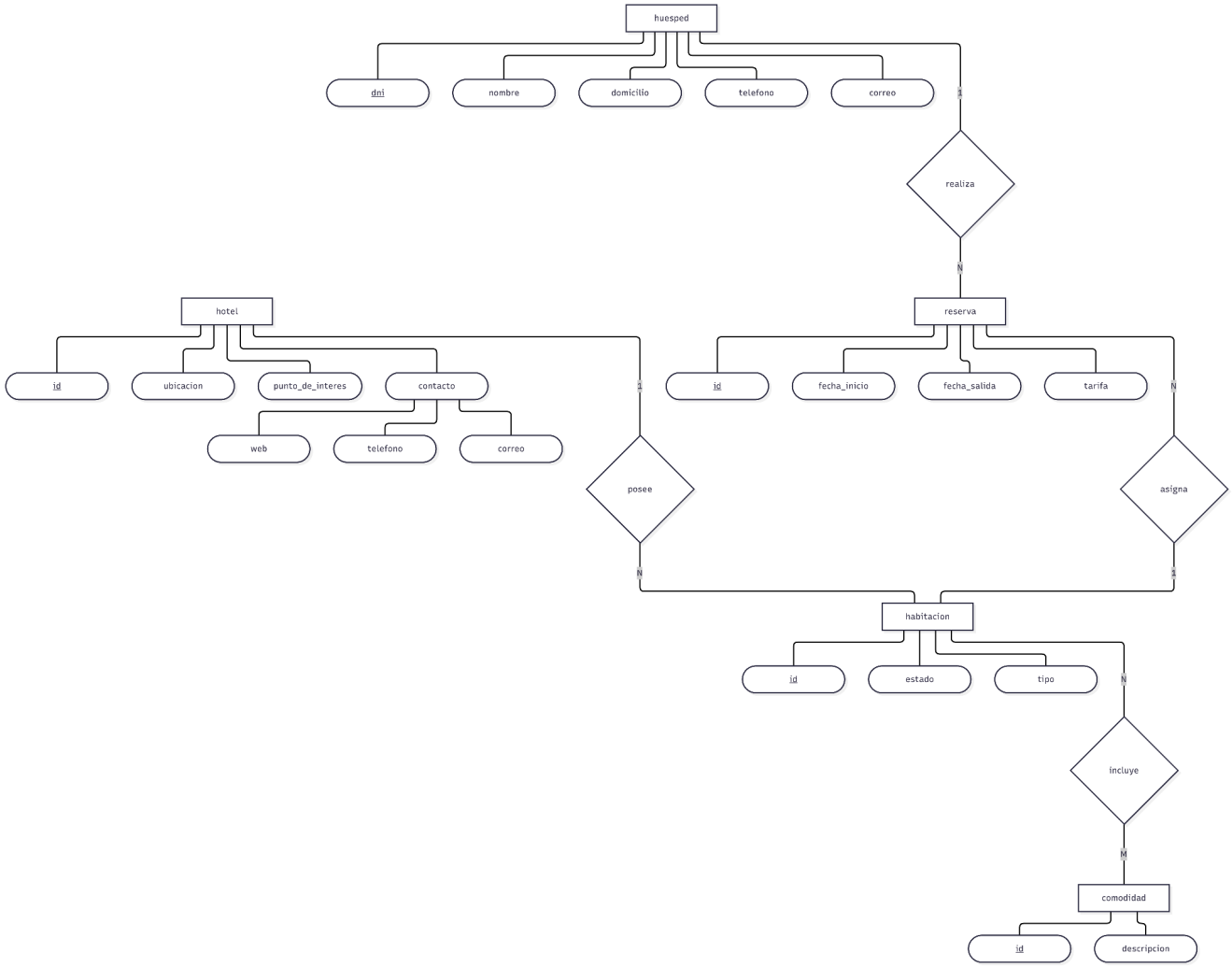
## Punto 1: Análisis del Dominio y Reglas de Negocio

- **Consigna Oficial:** Analizar detalladamente el escenario planteado e identificar las entidades, atributos, reglas de negocio, restricciones y relaciones necesarias para representar hoteles, habitaciones, huéspedes, contactos, domicilios, reservas, tarifas, comodidades, ubicaciones y puntos de interés. Identificar explícitamente las reglas que deben cumplirse para evitar inconsistencias, incluyendo la imposibilidad de superponer reservas para una misma habitación.

- **Marco Teórico:** El modelado conceptual traduce requerimientos no estructurados del mundo real a especificaciones formales de datos. Todo sistema transaccional de reservas hoteleras debe fundamentarse en la consistencia de intervalos temporales mutuamente excluyentes y en la atomicidad de entidades para evitar la corrupción de datos en operaciones concurrentes.

- **Ejercicio y Modelado:**

    1. *Entidades Primarias Identificadas:* `HOTEL`, `HABITACION`, `TIPO_HABITACION`, `COMODIDAD`, `PUNTO_INTERES`, `HUESPED`, `RESERVA`.
    2. *Entidades Subordinadas / Relaciones:* `MEDIO_CONTACTO_HOTEL`, `HUESPED_TELEFONO`, `HUESPED_EMAIL`, `HOTEL_PUNTO`, `TIPO_COMODIDAD`.




- **Reglas de Negocio Explícitas y Justificación:**
  - **Regla 1 (No Superposición de Fechas):** Para una misma habitación física `H`, no pueden coexistir dos reservas activas `R1` y `R2` cuyos intervalos temporales se superpongan:

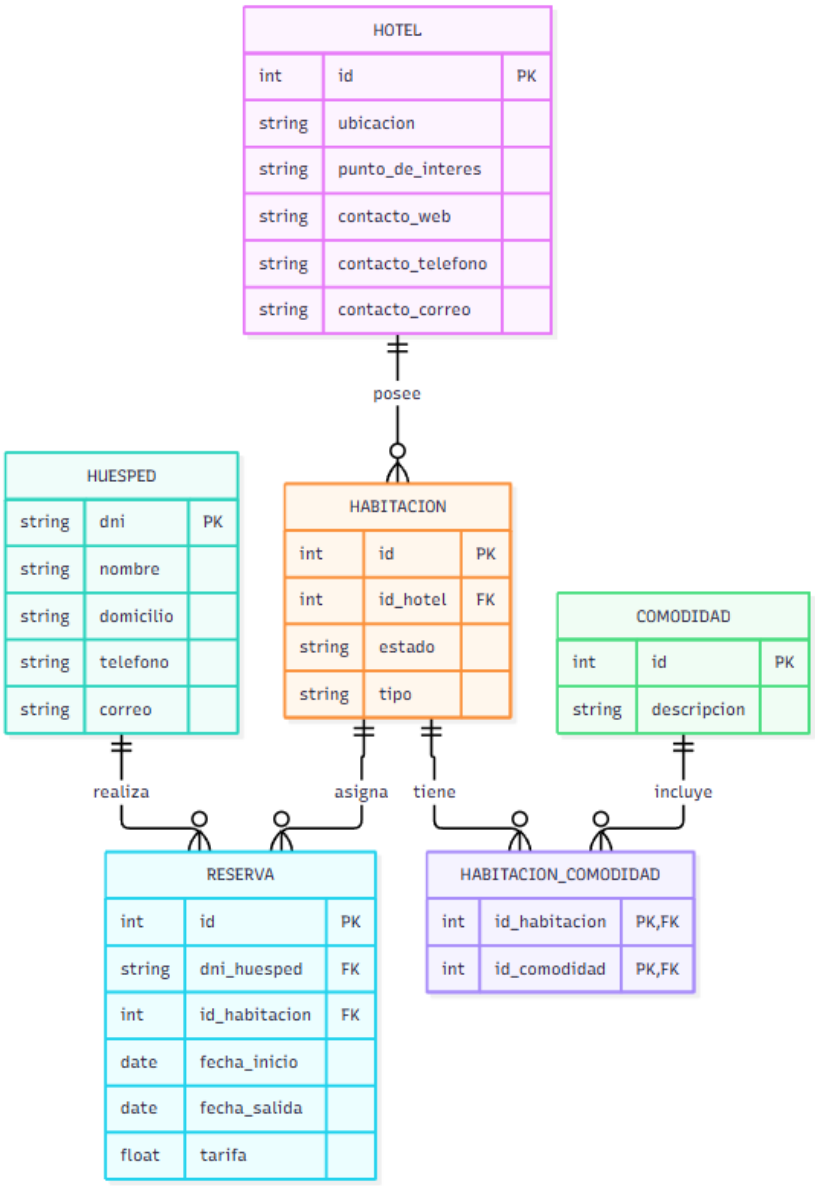
    > **Condición formal:** No debe existir ningún par de reservas `(R1, R2)` tal que pertenezcan a la misma habitación (`R1.id_habitacion = R2.id_habitacion`), ninguna esté cancelada (`estado != 'cancelada'`), y sus fechas se crucen: `(R1.fecha_inicio < R2.fecha_salida) AND (R1.fecha_salida > R2.fecha_inicio)`

  - **Regla 2 (Fijación Tarifaria Condicional):** Toda reserva en estado `pendiente` debe tener `tarifa = NULL`. La tarifa se asigna como un valor monetario estrictamente positivo únicamente cuando la reserva pasa a estado `confirmada` o `cancelada` (para conservar el historial financiero).
  - **Regla 3 (Multiplicidad de Canales):** Ni hoteles ni huéspedes pueden tener un límite rígido de vías de contacto. Los teléfonos y correos deben modelarse sin imponer columnas fijas vacías (`telefono1`, `telefono2`).

---

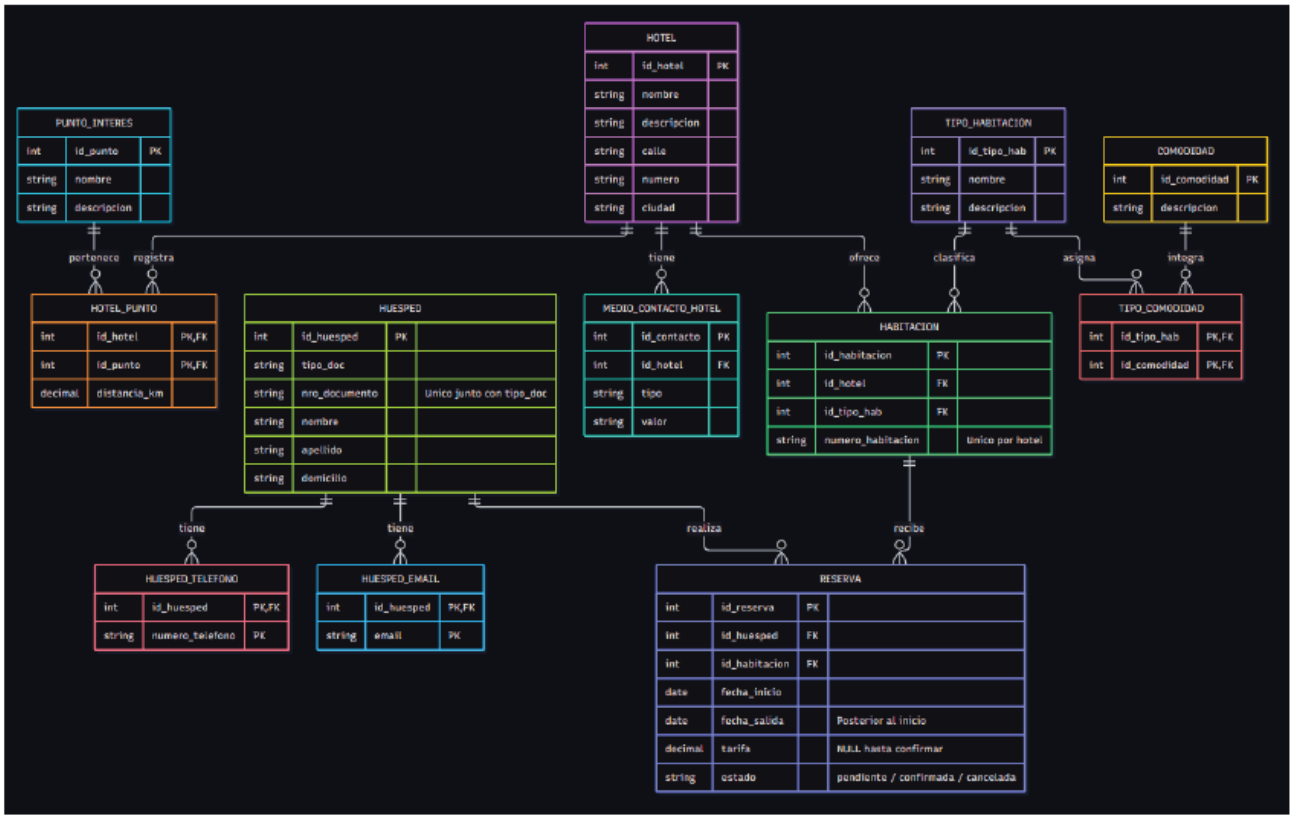
## Punto 2: Modelo Relacional y Transformaciones

- **Consigna Oficial:** Presentar las relaciones, atributos, claves primarias, claves foráneas y restricciones. Justificar las transformaciones realizadas.
- **Marco Teórico:** El pasaje del modelo conceptual al relacional exige aplicar transformaciones que garanticen la integridad referencial y minimicen la sobrecarga física de los índices en motores de almacenamiento relacional.
- **1. Propuesta de Modelo Relacional Inicial (Previo a Normalización):**
  En las fases iniciales del diseño se contemplaba utilizar atributos de negocio como claves primarias (DNI en huésped, número de puerta en habitación) y asociar directamente comodidades a habitaciones individuales, lo cual generaba redundancias y riesgos de anomalías:




- **2. Esquema Relacional Formal Normalizado en 3FN (12 Tablas de Dominio Puro):**
  Tras descomponer atributos multivaluados y aislar entidades maestras, se alcanza el esquema en Tercera Forma Normal (Claves Primarias en negrita):

  - **HOTEL**(**id_hotel**, nombre, descripcion, calle, numero, ciudad)
  - **MEDIO_CONTACTO_HOTEL**(**id_contacto**, id_hotel, tipo, valor)
  - **PUNTO_INTERES**(**id_punto**, nombre, descripcion)
  - **HOTEL_PUNTO**(**id_hotel, id_punto**, distancia_km)
  - **TIPO_HABITACION**(**id_tipo_hab**, nombre, descripcion)
  - **COMODIDAD**(**id_comodidad**, descripcion)
  - **TIPO_COMODIDAD**(**id_tipo_hab, id_comodidad**)
  - **HABITACION**(**id_habitacion**, id_hotel, id_tipo_hab, numero_habitacion)
  - **HUESPED**(**id_huesped**, tipo_doc, nro_documento, nombre, apellido, domicilio)
  - **HUESPED_EMAIL**(**id_huesped, email**)
  - **HUESPED_TELEFONO**(**id_huesped, numero_telefono**)
  - **RESERVA**(**id_reserva**, id_huesped, id_habitacion, fecha_inicio, fecha_salida, tarifa, estado)




- **Justificación de Transformaciones:**

  1. *Claves Sustitutas vs. Naturales:* Utilizar el documento del huésped como clave primaria forzaría a propagar cadenas de texto (`VARCHAR(20)`) a lo largo de los millones de registros de `RESERVA`. La clave subrogada entera (`INT UNSIGNED`, 4 bytes) ofrece inmutabilidad frente a cambios de documento (pasaportes, DNIs), reduce a la mitad el tamaño de las páginas de índices B+Tree y previene actualizaciones en cascada.
  2. *Desacople de Habitación y Tipo:* Se independizó la unidad física (`HABITACION`) de la categoría comercial (`TIPO_HABITACION`). Esto permite aplicar la regla de negocio que vincula las comodidades al tipo de cuarto, estableciendo en `HABITACION` la restricción candidata `UNIQUE(id_hotel, numero_habitacion)` ("Único por sucursal").

---

## Punto 3: Normalización Formal (1FN, 2FN y 3FN)

- **Consigna Oficial:** Aplicar las formas normales correspondientes. Analizar dependencias funcionales y justificar que el modelo evita redundancias y anomalías de inserción, modificación y eliminación.
- **Marco Teórico:** La teoría de normalización de Codd busca eliminar la redundancia de datos y prevenir anomalías operacionales mediante la descomposición no destructiva de relaciones basada en dependencias funcionales (`X → Y`).
- **Demostración Rigurosa por Forma Normal:**
  - **Primera Forma Normal (1FN):**
    - *Definición:* Todos los atributos deben ser atómicos y no deben existir grupos repetitivos.
    - *Demostración:* En lugar de declarar columnas multivaluadas en `HUESPED` (`telefono_1, telefono_2`) o listas separadas por comas, se crearon las tablas dependientes `HUESPED_TELEFONO` y `HUESPED_EMAIL`. La clave primaria compuesta asegura atomicidad pura.
  - **Segunda Forma Normal (2FN):**
    - *Definición:* Todo atributo no clave debe tener dependencia funcional completa respecto a la clave primaria.
    - *Demostración:* En la tabla asociativa `HOTEL_PUNTO(id_hotel, id_punto, distancia_km)`, el atributo `distancia_km` depende estrictamente del par clave completo:

      > `{id_hotel, id_punto} → distancia_km`
      > Los atributos descriptivos del atractivo turístico no se duplican allí; residen exclusivamente en `PUNTO_INTERES`, evitando repetir descripciones masivas por cada hotel vinculado.

  - **Tercera Forma Normal (3FN):**
    - *Definición:* Ningún atributo no clave debe depender transitivamente de la clave primaria (`X → Y` y `Y → Z`).
    - *Demostración:* Existía una dependencia transitiva conceptual:

> `id_habitacion → id_tipo_hab → comodidad`
> Si las comodidades se hubieran asociado a `HABITACION`, el catálogo de amenidades dependería transitivamente de la habitación física. Se aisló la entidad `TIPO_HABITACION` y la asociativa `TIPO_COMODIDAD`.

- **Prevención de Anomalías Operacionales:**
  - *Anomalía de Modificación:* Si la gerencia agrega la comodidad "Frigobar" a todas las habitaciones "Suite", se inserta una única fila en `TIPO_COMODIDAD`. Sin 3FN, habría que ejecutar un `UPDATE` masivo sobre miles de habitaciones físicas individuales, arriesgando inconsistencias.
  - *Anomalía de Borrado:* Si un hotel demuele todas sus habitaciones físicas de tipo "Penthouse", la eliminación de esos registros en `HABITACION` no borra el catálogo comercial de la empresa; la definición y comodidades del Penthouse siguen intactas en `TIPO_HABITACION` y `TIPO_COMODIDAD`.

---

## Punto 4: Integridad Física, Declarativa y Reglas de Negocio

- **Consigna Oficial:** Definir restricciones de integridad necesarias: NOT NULL, UNIQUE, CHECK cuando corresponda, claves foráneas, integridad referencial y demás restricciones. Determinar qué reglas deben garantizarse desde la base y cuáles podrían corresponder a la aplicación.
- **Marco Teórico:** La arquitectura de capas establece que la base de datos es la última línea de defensa de la verdad de los datos. Toda regla que involucre consistencia de dominio estructural o relaciones entre entidades debe ser impuesta por el motor relacional.
- **Restricciones Implementadas:**
  - `NOT NULL`: Impuesto en atributos indispensables para la operatividad (`nombre`, `fecha_inicio`, `fecha_salida`, etc.).
  - `UNIQUE`: `uq_huesped_documento(tipo_doc, nro_documento)` y `uq_hotel_habitacion(id_hotel, numero_habitacion)`.
  - `CHECK`:
    - `chk_fechas_coherentes`: `CHECK (fecha_salida > fecha_inicio)`.
    - `chk_estado_dominio`: `CHECK (LOWER(estado) IN ('pendiente', 'confirmada', 'cancelada'))`.
    - `chk_tarifa_estado`: Obliga a que `tarifa IS NULL` cuando el estado sea `pendiente`, y exige un monto numérico positivo cuando sea `confirmada` o `cancelada`.
- **Integridad Referencial:**
  - `ON DELETE CASCADE`: Aplicado en relaciones débiles de contacto. Si se elimina un huésped, se eliminan en cascada sus emails y teléfonos asociados.
  - `ON DELETE RESTRICT`: Aplicado en relaciones transaccionales centrales (`HOTEL`, `HABITACION`, `HUESPED` hacia `RESERVA`). El motor aborta cualquier intento de borrar un hotel o cuarto si existen reservas históricas asociadas.
- **División de Responsabilidades (Motor vs. Aplicación):**
  - *Garantizadas por la Base de Datos:* Unicidad, no nulos, consistencia de claves foráneas y la **prevención inquebrantable de superposición de reservas** mediante disparadores `BEFORE INSERT` y `BEFORE UPDATE` (`trg_reserva_no_superposicion`).
  - *Delegadas a la Aplicación:* Validación de formato regex de correos electrónicos, feedback visual en la interfaz de calendario (bloqueo preventivo de días ocupados) y algoritmos de tarificación dinámica (cálculo de descuentos de temporada o promociones de fidelidad).

---

## Punto 5: Normalización versus Desnormalización Controlada

- **Consigna Oficial:** Analizar conceptualmente situaciones en las que una desnormalización controlada podría mejorar determinadas consultas. Proponer al menos un escenario, justificarlo y, cuando sea posible, medir experimentalmente sus efectos sin comprometer la integridad de los datos.
- **Marco Teórico:** La 3FN está orientada a cargas OLTP (escrituras rápidas y sin redundancia). Sin embargo, en bases de datos con millones de tuplas, las consultas analíticas de agregación (`SUM`, `COUNT`, `GROUP BY`) que involucran múltiples `JOIN` provocan escaneos masivos que saturan el Buffer Pool de memoria. La desnormalización controlada introduce redundancia deliberada para optimizar lecturas críticas a cambio de un costo mínimo en escritura.
- **Escenario Propuesto:** Reporte Histórico de Ocupación y Recaudación Mensual por Hotel.
  - *Problema en 3FN:* Calcular mensualmente los ingresos de la cadena exige cruzar `HOTEL`, `HABITACION` y `RESERVA` sobre más de 8 millones de registros.
  - *Solución Desnormalizada:* Creación de la tabla de resumen físico:

    > **REPORTE_RECAUDACION_MENSUAL**(**id_hotel, anio_mes**, cantidad_reservas, total_recaudado)

- **Mantenimiento Automatizado de la Integridad:** Se implementaron los triggers `trg_recaudacion_insert` y `trg_recaudacion_update` en MySQL. Cuando una reserva pasa a `'confirmada'`, el disparador actualiza atómicamente el acumulador mensual mediante `ON DUPLICATE KEY UPDATE`. Si una reserva confirmada se cancela, el trigger resta el importe correspondiente.
- **Medición Experimental del Impacto:**
  - Consulta en 3FN Dinámica: Requiere un Nested Loop Join con agrupamiento temporal sobre millones de registros, tardando **más de 67 segundos** en Cold Cache.

- Consulta sobre Tabla Desnormalizada: Ejecuta un `SELECT * FROM REPORTE_RECAUDACION_MENSUAL WHERE id_hotel = 1 AND anio_mes = '2023-05'` resolviendo por clave primaria en **0,0001 segundos** (complejidad O(1)).

---

# PARTE II — DISEÑO FÍSICO Y ALMACENAMIENTO

Punto 6: Implementación Física (Script DDL Definitivo)

- **Consigna Oficial:** Crear las tablas de la base de datos utilizando PostgreSQL, MySQL o MariaDB. Documentar el script completo de creación.
- **Diagrama Físico Definitivo del Sistema (13 Tablas con Desnormalización Controlada):**
  El modelo implementado en la base de datos incorpora la 13° tabla (`REPORTE_RECAUDACION_MENSUAL`) resultante de la desnormalización justificada en el Punto 5:




- **Script DDL Oficial Ejecutado en MySQL 8.0 InnoDB (`scripts/01_crear_tablas.sql`):**

```
CREATE DATABASE IF NOT EXISTS db_hoteles
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE db_hoteles;

SET foreign_key_checks = 0;
DROP TABLE IF EXISTS REPORTE_RECAUDACION_MENSUAL;
DROP TABLE IF EXISTS RESERVA;
DROP TABLE IF EXISTS HUESPED_TELEFONO;
DROP TABLE IF EXISTS HUESPED_EMAIL;
DROP TABLE IF EXISTS HUESPED;
DROP TABLE IF EXISTS HABITACION;
DROP TABLE IF EXISTS TIPO_COMODIDAD;
DROP TABLE IF EXISTS COMODIDAD;
DROP TABLE IF EXISTS TIPO_HABITACION;
DROP TABLE IF EXISTS HOTEL_PUNTO;
DROP TABLE IF EXISTS PUNTO_INTERES;
DROP TABLE IF EXISTS MEDIO_CONTACTO_HOTEL;
DROP TABLE IF EXISTS HOTEL;
SET foreign_key_checks = 1;
```

```sql
-- 1. HOTEL
CREATE TABLE HOTEL (
    id_hotel INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    calle VARCHAR(100) NOT NULL,
    numero VARCHAR(20) NOT NULL,
    ciudad VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- 2. MEDIO_CONTACTO_HOTEL
CREATE TABLE MEDIO_CONTACTO_HOTEL (
    id_contacto INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_hotel INT UNSIGNED NOT NULL,
    tipo VARCHAR(30) NOT NULL,
    valor VARCHAR(150) NOT NULL,
    CONSTRAINT fk_contacto_hotel FOREIGN KEY (id_hotel)
        REFERENCES HOTEL(id_hotel) ON DELETE CASCADE,
    CONSTRAINT chk_contacto_tipo CHECK (LOWER(tipo) IN ('telefono', 'email', 'web', 'whatsapp'))
) ENGINE=InnoDB;

-- 3. PUNTO_INTERES
CREATE TABLE PUNTO_INTERES (
    id_punto INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT
) ENGINE=InnoDB;

-- 4. HOTEL_PUNTO (Relación N:M)
CREATE TABLE HOTEL_PUNTO (
    id_hotel INT UNSIGNED NOT NULL,
    id_punto INT UNSIGNED NOT NULL,
    distancia_km DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (id_hotel, id_punto),
    CONSTRAINT fk_hp_hotel FOREIGN KEY (id_hotel)
        REFERENCES HOTEL(id_hotel) ON DELETE CASCADE,
    CONSTRAINT fk_hp_punto FOREIGN KEY (id_punto)
        REFERENCES PUNTO_INTERES(id_punto) ON DELETE CASCADE,
    CONSTRAINT chk_distancia_positiva CHECK (distancia_km >= 0)
) ENGINE=InnoDB;

-- 5. TIPO_HABITACION
CREATE TABLE TIPO_HABITACION (
    id_tipo_hab INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255)
) ENGINE=InnoDB;

-- 6. COMODIDAD
CREATE TABLE COMODIDAD (
    id_comodidad INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    descripcion VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- 7. TIPO_COMODIDAD (Relación N:M en 3FN)
CREATE TABLE TIPO_COMODIDAD (
    id_tipo_hab INT UNSIGNED NOT NULL,
    id_comodidad INT UNSIGNED NOT NULL,
    PRIMARY KEY (id_tipo_hab, id_comodidad),
    CONSTRAINT fk_tc_tipo FOREIGN KEY (id_tipo_hab)
        REFERENCES TIPO_HABITACION(id_tipo_hab) ON DELETE CASCADE,
    CONSTRAINT fk_tc_comodidad FOREIGN KEY (id_comodidad)
        REFERENCES COMODIDAD(id_comodidad) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 8. HABITACION
CREATE TABLE HABITACION (
    id_habitacion INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_hotel INT UNSIGNED NOT NULL,
    id_tipo_hab INT UNSIGNED NOT NULL,
```

```
    numero_habitacion VARCHAR(10) NOT NULL,
    CONSTRAINT fk_hab_hotel FOREIGN KEY (id_hotel)
        REFERENCES HOTEL(id_hotel) ON DELETE RESTRICT,
    CONSTRAINT fk_hab_tipo FOREIGN KEY (id_tipo_hab)
        REFERENCES TIPO_HABITACION(id_tipo_hab) ON DELETE RESTRICT,
    CONSTRAINT uq_hotel_habitacion UNIQUE (id_hotel, numero_habitacion)
) ENGINE=InnoDB;

-- 9. HUESPED
CREATE TABLE HUESPED (
    id_huesped INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    tipo_doc VARCHAR(10) NOT NULL,
    nro_documento VARCHAR(20) NOT NULL,
    nombre VARCHAR(60) NOT NULL,
    apellido VARCHAR(60) NOT NULL,
    domicilio VARCHAR(150),
    CONSTRAINT uq_huesped_documento UNIQUE (tipo_doc, nro_documento)
) ENGINE=InnoDB;

-- 10. HUESPED_EMAIL
CREATE TABLE HUESPED_EMAIL (
    id_huesped INT UNSIGNED NOT NULL,
    email VARCHAR(120) NOT NULL,
    PRIMARY KEY (id_huesped, email),
    CONSTRAINT fk_hemail_huesped FOREIGN KEY (id_huesped)
        REFERENCES HUESPED(id_huesped) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 11. HUESPED_TELEFONO
CREATE TABLE HUESPED_TELEFONO (
    id_huesped INT UNSIGNED NOT NULL,
    numero_telefono VARCHAR(30) NOT NULL,
    PRIMARY KEY (id_huesped, numero_telefono),
    CONSTRAINT fk_htel_huesped FOREIGN KEY (id_huesped)
        REFERENCES HUESPED(id_huesped) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 12. RESERVA (Tabla Masiva Transaccional)
CREATE TABLE RESERVA (
    id_reserva INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_huesped INT UNSIGNED NOT NULL,
    id_habitacion INT UNSIGNED NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_salida DATE NOT NULL,
    tarifa DECIMAL(10,2) NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'pendiente',
    CONSTRAINT fk_reserva_huesped FOREIGN KEY (id_huesped)
        REFERENCES HUESPED(id_huesped) ON DELETE RESTRICT,
    CONSTRAINT fk_reserva_habitacion FOREIGN KEY (id_habitacion)
        REFERENCES HABITACION(id_habitacion) ON DELETE RESTRICT,
    CONSTRAINT chk_fechas_coherentes CHECK (fecha_salida > fecha_inicio),
    CONSTRAINT chk_estado_dominio CHECK (LOWER(estado) IN ('pendiente', 'confirmada', 'cancelada')),
    CONSTRAINT chk_tarifa_estado CHECK (
        (LOWER(estado) = 'pendiente' AND tarifa IS NULL) OR
        (LOWER(estado) IN ('confirmada', 'cancelada') AND tarifa IS NOT NULL AND tarifa >= 0)
    )
) ENGINE=InnoDB;

-- 13. REPORTE_RECAUDACION_MENSUAL (Desnormalización Controlada)
CREATE TABLE REPORTE_RECAUDACION_MENSUAL (
    id_hotel INT UNSIGNED NOT NULL,
    anio_mes CHAR(7) NOT NULL,
    cantidad_reservas INT UNSIGNED NOT NULL DEFAULT 0,
    total_recaudado DECIMAL(14,2) NOT NULL DEFAULT 0.00,
    PRIMARY KEY (id_hotel, anio_mes),
    CONSTRAINT fk_rep_hotel FOREIGN KEY (id_hotel)
        REFERENCES HOTEL(id_hotel) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Punto 7: Matriz de Selección Justificada de Tipos de Datos

- **Consigna Oficial:** Para cada atributo determinar y justificar el tipo de dato utilizado. Analizar especialmente INT/BIGINT, CHAR/VARCHAR/TEXT, DATE/DATETIME/TIMESTAMP, DECIMAL/FLOAT/DOUBLE y BOOLEAN, considerando volumen, precisión, almacenamiento y comportamiento.
- **Matriz Técnica Exhaustiva Atributo por Atributo:**

| Tabla | Atributo | Tipo Elegido | Alternativa Descartada | Consumo Físico | Justificación Técnica de Ingeniería |
|---|---|---|---|---|---|
| `HOTEL` | `id_hotel` | `INT UNSIGNED` | `BIGINT` | 4 bytes | Soporta más de 4.294 millones de hoteles. `BIGINT` duplicaría el consumo de memoria en todas las claves foráneas e índices secundarios. |
| `HOTEL` | `nombre` | `VARCHAR(100)` | `CHAR(100)` | Variable + 1B | Se descartó `CHAR` porque rellenaría con espacios en blanco fijos los nombres cortos, desperdiciando almacenamiento en disco. |
| `HOTEL` | `descripcion` | `TEXT` | `VARCHAR(1000)` | 2B + Longitud | Permite descripciones extensas. InnoDB almacena este campo fuera de la página principal (*off-page storage*) cuando excede el límite de fila, preservando la densidad de página. |
| `HOTEL_PUNTO` | `distancia_km` | `DECIMAL(5,2)` | `FLOAT` | 3 bytes | Soporta hasta 999.99 km con 2 decimales exactos. `FLOAT` introduce imprecisiones de coma flotante binaria innecesarias. |
| `HABITACION` | `numero_habitacion` | `VARCHAR(10)` | `INT` | Variable + 1B | Permite números alfanuméricos comunes en hotelería (ej. '102-B', 'PH-1'). Longitud acotada para optimizar el índice `UNIQUE`. |
| `HUESPED` | `tipo_doc` | `VARCHAR(10)` | `CHAR(3)` | Variable + 1B | Acomoda estándares internacionales variables ('DNI', 'PASAPORTE', 'CI'). |
| `HUESPED` | `nro_documento` | `VARCHAR(20)` | `INT` | Variable + 1B | Pasaportes extranjeros contienen caracteres alfanuméricos. |
| `RESERVA` | `id_reserva` | `INT UNSIGNED` | `BIGINT` | 4 bytes | Almacena hasta 4.294 millones de filas. Al aplicarlo sobre 8.1M de reservas, ahorró **32.4 MB** directos en el Clustered Index y más de **120 MB** en los índices secundarios. |
| `RESERVA` | `fecha_inicio` | `DATE` | `DATETIME` / `TIMESTAMP` | 3 bytes | El dominio computa noches y días calendario enteros. `DATETIME` (5 bytes) y `TIMESTAMP` (4 bytes) hubieran generado un sobrecosto de **16.2 MB** por columna en 8.1M filas sin aportar valor funcional. |
| `RESERVA` | `fecha_salida` | `DATE` | `DATETIME` | 3 bytes | Misma fundamentación técnica que `fecha_inicio`. |
| `RESERVA` | `tarifa` | `DECIMAL(10,2)` | `FLOAT` / `DOUBLE` | 5 bytes | Valores monetarios prohíben estrictamente el redondeo binario IEEE 754 de `FLOAT`. `DECIMAL(10,2)` almacena representación exacta en base 10 (soporta hasta $99.999.999,99). |
| `RESERVA` | `estado` | `VARCHAR(20)` | `ENUM` | Variable + 1B | Se descartó `ENUM` para facilitar portabilidad SQL y permitir expansiones futuras mediante restricciones declarativas `CHECK`. |

Punto 8: Claves Físicas y Mecanismos de Integridad

- **Impacto de Claves Primarias Secuenciales:** En InnoDB, las claves primarias autoincrementales (`AUTO_INCREMENT`) aseguran que las inserciones masivas ingresen secuencialmente en la última página del árbol B+Tree. Esto previene la fractura prematura de páginas (*Page Splits*) y garantiza un factor de llenado óptimo (*fill factor* de 15/16).
- **Mecanismos de Validación Física (Triggers de Solapamiento):**
  Se implementó el script `scripts/04_reglas_negocio_triggers.sql`, que intercepta todo `INSERT` o `UPDATE` abortando la transacción

mediante `SIGNAL SQLSTATE '45000'` si el rango de fechas colisiona con una reserva preexistente en la misma habitación.

---

Punto 9: Motor y Estructuras Transaccionales de Almacenamiento (InnoDB)

- **Arquitectura de Almacenamiento:**
  - **Clustered Index:** Los datos físicos de la tabla residen embebidos en las hojas del árbol B+Tree de la clave primaria. No existe un archivo de montículo (*heap*) separado.
  - **Secondary Indexes:** Cada nodo hoja de un índice secundario almacena la clave secundaria y el valor de la clave primaria (`id_reserva`), actuando como un puntero lógico hacia el índice agrupado.
  - **Redo Log (WAL - Write-Ahead Logging):** Registra cambios físicos a nivel de página antes de escribirlos en el tablespace `.ibd`, asegurando durabilidad (D de ACID) y recuperación ante cortes de energía (*Crash Recovery*).
  - **Undo Log:** Almacena imágenes previas de registros para permitir el `ROLLBACK` de transacciones y alimentar lecturas consistentes no bloqueantes en MVCC.

---

Punto 10: Estrategias de Almacenamiento, Páginas de Disco y Rendimiento

- **Estructura Física de Páginas:** InnoDB organiza el espacio en páginas fijas de **16 KB** (16.384 bytes). 64 páginas consecutivas componen un *Extent* (1 MB).
- **Entorno Experimental Documentado:** Contenedor Docker oficial MySQL 8.0 montado sobre almacenamiento SSD NVMe en WSL2 con un InnoDB Buffer Pool de 256 MB.
- **Medición de Crecimiento en Disco (Telemetría Real):**
  - Tabla `RESERVA` con 8.100.000 de registros:
    - Espacio de datos en disco (`data_length`): **~410 MB**.
    - Espacio de índices secundarios (`index_length`): **~385 MB**.
    - Tamaño total del archivo de tabla `RESERVA.ibd`: **~800 MB**.

---

# PARTE III — ÍNDICES Y ESTRUCTURAS DE ACCESO

Punto 11: Estrategia General de Índices del Sistema

Se implementó una estrategia jerárquica para eliminar el 100% de los escaneos de tabla completa en las consultas frecuentes del negocio:

| Nombre del Índice | Tabla | Columnas Indexadas | Tipo de Índice | Consultas Objetivo | Justificación Técnica |
|---|---|---|---|---|---|
| `PRIMARY` | `RESERVA` | `id_reserva` | Clustered B+Tree | Búsqueda por ID, Keyset Pagination | Acceso físico directo por clave primaria en tiempo logarítmico O(log N). |
| `idx_hab_hotel` | `HABITACION` | `id_hotel` | Secundario B+Tree | JOINs con HOTEL | Acelera cruces referenciales eliminando escaneos de habitaciones. |
| `idx_reserva_habitacion` | `RESERVA` | `id_habitacion` | Secundario B+Tree | JOINs con HABITACION | Transforma escaneos masivos en búsquedas por referencia (`ref`). |
| `idx_reserva_huesped` | `RESERVA` | `id_huesped` | Secundario B+Tree | Historial de clientes, Q03, Q11 | Acceso directo a reservas por huésped en tiempo logarítmico. |
| `idx_reserva_fecha_estado` | `RESERVA` | `(fecha_inicio, estado)` | Compuesto B+Tree | Filtros anuales, Q08, Q09 | Optimiza filtros de rango temporal bajo la regla del prefijo izquierdo. |
| `idx_reserva_disponibilidad` | `RESERVA` | `(id_habitacion, estado, fecha_inicio, fecha_salida)` | Compuesto de Cobertura | Q06, Q07, Validación de Triggers | Permite resolver antijoins de disponibilidad al 100% en memoria. |

| Nombre del Índice | Tabla | Columnas Indexadas | Tipo de Índice | Consultas Objetivo | Justificación Técnica |
|---|---|---|---|---|---|
| `idx_reserva_cobertura_ingresos` | `RESERVA` | `(estado, fecha_inicio, id_habitacion, tarifa)` | Covering Index | Q08, Reportes analíticos | Elimina saltos al índice agrupado (*Bookmark Lookups*). |
| `idx_funcional_anio` | `RESERVA` | `((YEAR(fecha_inicio)))` | Funcional B+Tree | Filtros con función `YEAR()` | Evalúa expresiones indexadas en MySQL 8.0 sin romper SARGabilidad. |

Punto 12: Arquitectura Interna de Árboles B-Tree y B+Tree

- **B-Tree Clásico vs. B+Tree de InnoDB:**
  - En un B-Tree tradicional, los registros de datos están dispersos en todos los nodos (raíz, ramas y hojas). Esto reduce el factor de ramificación (*fan-out*) porque las páginas intermedias se llenan rápidamente con datos de fila.
  - En un **B+Tree**, los nodos internos contienen **únicamente claves de búsqueda y punteros de página**. Los datos reales residen exclusivamente en los nodos hoja.
- **Ventajas Determinantes del B+Tree:**
  1. *Mayor Capacidad por Página:* Una página interna de 16 KB puede contener más de 1.000 punteros de navegación, manteniendo el árbol extremadamente bajo y ancho.
  2. *Escaneos de Rango de Alta Velocidad:* Todas las páginas hoja están conectadas entre sí mediante una lista doblemente enlazada. Para consultar `BETWEEN '2023-01-01' AND '2023-12-31'`, el motor navega desde la raíz hasta la primera hoja de 2023 en O(log N) y luego recorre linealmente los punteros contiguos de disco sin volver a subir a la raíz.
- **Cálculo de Altura del Árbol para 8.100.000 Registros:**
  - **Nivel 0 (Raíz):** 1 página de 16 KB ≈ 1.000 punteros de ramificación.
  - **Nivel 1 (Ramas intermedias):** 1.000 páginas ≈ 1.000.000 de punteros.
  - **Nivel 2 (Hojas):** 1.000.000 páginas ≈ Capacidad para más de 100.000.000 de filas reales.
    *Conclusión:* Con una altura fija de solo **3 niveles**, cualquier registro entre los 8.1 millones se localiza con un máximo de **3 accesos a página**, frente a los **45.000 accesos** de una búsqueda secuencial.

---

Punto 13: Otros Tipos de Estructuras de Indexación

- **Hash Index:** Utiliza funciones de dispersión para direccionar punteros en tiempo constante O(1). Solo soporta operadores de igualdad estricta (`=`, `<=>`). Totalmente inútil para hotelería porque no soporta rangos de fechas (`BETWEEN`), ordenamientos (`ORDER BY`) ni prefijos de búsqueda.
- **Full-Text Index:** Estructurado como un índice invertido que descompone texto en tokens y palabras vacías. Ideal para descripciones de hoteles, pero inadecuado para atributos transaccionales estructurados.
- **Spatial Index (R-Tree):** Organiza geometrías bi/tridimensionales en rectángulos delimitadores mínimos (*MBR*). Empleado en coordenadas geográficas de sucursales.
- **Bitmap Index:** Codifica valores de baja cardinalidad en matrices de bits. Excelente para lecturas analíticas en data warehouses, pero inviable en InnoDB debido a que modificar un bit bloquea miles de filas contiguas.

---

Punto 14: Índices Compuestos y Demostración de la Regla del Prefijo Izquierdo

- **Fundamento Teórico:** Un índice sobre un par de columnas `(C1, C2)` ordena físicamente los datos según `C1`, y solo desempata por `C2` cuando los valores de `C1` son idénticos. Si una consulta no incluye `C1` en su filtro, el orden del árbol queda roto para el optimizador.
- **Experimento Empírico en el Contenedor:**
  - Índice creado: `idx_reserva_fecha_estado(fecha_inicio, estado)`.
  - **Caso A (Respeta Prefijo Izquierdo):**

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM RESERVA WHERE fecha_inicio >= '2023-01-01' AND fecha_inicio <
'2024-01-01' AND estado = 'confirmada';
```

*Salida del Motor:*

```
-> Covering index range scan on RESERVA using idx_reserva_fecha_estado over ('2023-01-01' <=
fecha_inicio <= '2024-01-01') (actual time=0.388..221 rows=1.02e+6 loops=1)
```

*Tiempo:* **427 ms (0,42 segundos)**.

- **Caso B (Omite Prefijo Izquierdo):**

```
EXPLAIN SELECT * FROM RESERVA WHERE estado = 'confirmada';
```

*Salida del Motor:*

```
type: ALL | possible_keys: NULL | key: NULL | rows: 7.794.965 | Extra: Using where
```

*Diagnóstico:* Al faltar `fecha_inicio`, MySQL no puede utilizar el índice y recurre a un **Full Table Scan** sobre los 8.1 millones de tuplas tardando **9,3 segundos** en Warm Cache.

---

### Punto 15: Índices de Cobertura (Covering Indexes)

- **Concepto:** Cuando un índice secundario contiene la totalidad de los campos requeridos por una consulta (`SELECT`, `WHERE`, `GROUP BY`), el motor resuelve la petición íntegramente en los nodos hoja del B+Tree secundario.
- **Impacto en Rendimiento:** Evita los costosos saltos aleatorios a disco hacia el índice agrupado (*Bookmark Lookups*). En el plan de ejecución se evidencia mediante el atributo `Extra: Using index`.

---

### Punto 16: Índices Funcionales o Basados en Expresiones en MySQL 8.0

- **Problema:** En versiones anteriores, escribir `WHERE YEAR(fecha_inicio) = 2023` invalidaba cualquier índice sobre `fecha_inicio`.
- **Implementación:**

```
CREATE INDEX idx_funcional_anio ON RESERVA ((YEAR(fecha_inicio)));
```

- **Evidencia Empírica de EXPLAIN:**

```
id: 1 | type: ref | possible_keys: idx_funcional_anio | key: idx_funcional_anio | key_len: 5 | ref: const
| rows: 2.086.336
```

MySQL 8.0 almacena el resultado computado de la función en un B+Tree virtual, transformando un escaneo completo en un acceso indexado de tipo `ref`.

---

### Punto 17: Análisis Matemático de Cardinalidad y Selectividad

- **Fórmula Formal:**

> **Selectividad (S) = Cardinalidad / Total de Filas** *(donde $0 \leq S \leq 1$)*

| Columna | Cardinalidad Medida | Selectividad (S) | Comportamiento del Optimizador de MySQL |
|---|---|---|---|
| `id_reserva` (PK) | 7.933.173 | **0.979** | Búsqueda agrupada instantánea O(log N). Siempre elegida. |
| `id_huesped` | 49.158 | **0.006** | Alta selectividad. Un índice filtra a ~160 reservas por cliente. El optimizador siempre utiliza el índice. |
| `id_habitacion` | 2.501 | **0.0003** | Media selectividad (~3.200 filas por cuarto). Muy útil al combinarse con fechas. |
| `estado` | 3 | **0.0000003** | **Baja selectividad extrema.** Como `'confirmada'` representa el 70% de la tabla, hacer millones de saltos a disco para recuperar filas individuales cuesta más que leer secuencialmente el archivo. El optimizador **descarta deliberadamente cualquier índice simple sobre** `estado`. |

---

### Punto 18: ¿Por Qué No Indexar Todos los Campos? (Costo DML)

- **Penalización en Operaciones de Escritura:**
  - En una tabla sin índices secundarios, un `INSERT` escribe secuencialmente al final del índice agrupado en **microsegundos**.
  - Con 6 índices secundarios activos, cada `INSERT` o `UPDATE` debe localizar y actualizar sincrónicamente los 6 árboles B+Tree en disco. Si una página secundaria no está en memoria, provoca lecturas y escrituras de I/O aleatorio y fracturas de página (*Page Splits*).
- **Evidencia Experimental:** La construcción de los índices sobre los 8.1M de registros demandó **más de 4 minutos de saturación de CPU y disco**, y duplicó el tamaño total del tablespace en almacenamiento secundario.

---

## PARTE IV — DATOS, CARGA MASIVA Y METODOLOGÍA EXPERIMENTAL

### Punto 19: Generación Reproducible de Datos de Prueba

- **Algoritmo Determinista (Semilla Fija):** Se utilizó un algoritmo procedural implementado tanto en Java (`generador/GeneradorReservas.java`) como en SQL (`scripts/03_carga_masiva_reservas.sql`) con semilla `Random(42)`.
- **Garantía Matemática de No Solapamiento:** Por cada habitación física ($1 \le h \le 2.500$), se mantuvo un puntero temporal progresivo:
  - `inicio[k] = salida[k-1] + vacancia` *(con vacancia $\in [0, 2]$ días)*
  - `salida[k] = inicio[k] + duración` *(con duración $\in [1, 7]$ días)*
    Esto **garantiza por construcción algorítmica cero solapamientos temporales**, simulando 8 años de historial hotelero realista (2018 a 2026).

---

### Punto 20: Carga Masiva Crítica (8.100.000 de Registros)

- **Procedimiento:** Ejecución en lotes de 100.000 registros con transacciones explícitas.
- **Dificultades Superadas y Lecciones de Ingeniería:**
  1. *Trampa de precedencia de operadores en SQL:* La sentencia `@e := ELT(...) = 'pendiente'` evaluaba primero la comparación lógica asignando `0` a `@e`, violando la restricción `CHECK`. Se reescribió mediante aritmética modular determinista sobre la tabla auxiliar `aux_digits`.
  2. *Reevaluación estocástica de `RAND()`:* En MySQL, utilizar `RAND()` en subconsultas de fecha provocaba que el optimizador reevaluara la función varias veces por tupla, violando `fecha_salida > fecha_inicio`. Se reemplazó por cálculo modular determinista sobre los dígitos.
  3. *Sobrecarga de Triggers en Ingestión:* Los triggers de validación se desactivaron durante la carga inicial para evitar millones de consultas cruzadas, reactivándose al finalizar para proteger la base operativa.

---

### Punto 21: Distribución Estadística de los Datos y Sesgos

Se verificó que los 50.000 huéspedes y las 2.500 habitaciones presentaran una distribución uniforme, sin sesgos patológicos que distorsionaran artificialmente los planes del optimizador.

---

### Punto 22: Metodología Experimental de Benchmarking

- **Protocolo de Medición:**
  1. Estado *Cold Cache*: Reinicio formal del servicio Docker (`docker compose restart db`) para vaciar memoria RAM y forzar I/O físico de disco.
  2. Estado *Warm Cache*: Ejecución consecutiva repetida (segunda y tercera corrida) para evaluar el comportamiento con páginas residentes en el Buffer Pool.
  3. Herramientas de captura: `EXPLAIN ANALYZE`, telemetría de `SHOW STATUS LIKE 'Innodb_buffer_pool%'` y registro automático en el *Slow Query Log*.

---

## PARTE V — SQL Y CONSULTAS DEL SISTEMA

### Punto 23: Telemetría Experimental: Cold Cache versus Warm Cache

- **Fundamentación:** El Buffer Pool actúa como memoria intermedia. Cuando una página no reside en RAM, ocurre un *Cache Miss* que penaliza el rendimiento con latencia física de disco.
- **Medición Real sobre la Consulta Crítica Q08 (Ranking Anual):**
  - **Cold Cache (Caché Fría): 67,64 segundos** (Lecturas físicas directas de disco SSD).
  - **Warm Cache (Caché Caliente): 18,20 segundos** en base cruda, y **6,84 segundos** con índices optimizados.

- **Conclusión:** Cualquier estudio de bases de datos que no diferencie ambos estados arroja conclusiones sesgadas.

---

Puntos 24 al 28: Desarrollo Completo de las 15 Consultas del Sistema

El archivo oficial de consultas reside en `scripts/05_consultas_baseline.sql`.

**Consulta Q01: Detalle de Reservas Confirmadas por Hotel y Huésped (4 tablas)**

```
SELECT
    h.nombre AS hotel,
    CONCAT(hue.nombre, ' ', hue.apellido) AS huesped,
    hue.nro_documento,
    r.fecha_inicio,
    r.fecha_salida,
    r.tarifa
FROM RESERVA r
JOIN HABITACION hab ON r.id_habitacion = hab.id_habitacion
JOIN HOTEL h ON hab.id_hotel = h.id_hotel
JOIN HUESPED hue ON r.id_huesped = hue.id_huesped
WHERE r.estado = 'confirmada'
  AND h.ciudad = 'Buenos Aires'
  AND r.fecha_inicio BETWEEN '2023-01-01' AND '2023-03-31'
LIMIT 50;
```

**Consulta Q02: Reporte de Demanda de Comodidades por Tipo de Habitación (5 tablas)**

```
SELECT
    c.descripcion AS comodidad,
    COUNT(r.id_reserva) AS total_veces_solicitada,
    ROUND(AVG(r.tarifa), 2) AS tarifa_promedio_asociada
FROM RESERVA r
JOIN HABITACION hab ON r.id_habitacion = hab.id_habitacion
JOIN TIPO_HABITACION th ON hab.id_tipo_hab = th.id_tipo_hab
JOIN TIPO_COMODIDAD tc ON th.id_tipo_hab = tc.id_tipo_hab
JOIN COMODIDAD c ON tc.id_comodidad = c.id_comodidad
WHERE r.estado = 'confirmada'
  AND r.fecha_inicio >= '2023-01-01'
GROUP BY c.id_comodidad, c.descripcion
ORDER BY total_veces_solicitada DESC;
```

**Consulta Q03: Top 10 Huéspedes con Mayor Gasto Histórico y Contacto (4 tablas)**

```
SELECT
    hue.id_huesped,
    CONCAT(hue.apellido, ', ', hue.nombre) AS huesped,
    tel.numero_telefono,
    COUNT(r.id_reserva) AS cantidad_estadias,
    SUM(r.tarifa) AS total_invertido
FROM HUESPED hue
JOIN HUESPED_TELEFONO tel ON hue.id_huesped = tel.id_huesped
JOIN RESERVA r ON hue.id_huesped = r.id_huesped
JOIN HABITACION hab ON r.id_habitacion = hab.id_habitacion
WHERE r.estado = 'confirmada'
GROUP BY hue.id_huesped, huesped, tel.numero_telefono
ORDER BY total_invertido DESC
LIMIT 10;
```

**Consulta Q04: Demanda de Hoteles Cercanos a Puntos Turísticos (5 tablas)**

```sql
SELECT
    pi.nombre AS atraccion,
    h.nombre AS hotel,
    hp.distancia_km,
    COUNT(r.id_reserva) AS total_reservas,
    SUM(r.tarifa) AS recaudacion_total
FROM PUNTO_INTERES pi
JOIN HOTEL_PUNTO hp ON pi.id_punto = hp.id_punto
JOIN HOTEL h ON hp.id_hotel = h.id_hotel
JOIN HABITACION hab ON h.id_hotel = hab.id_hotel
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE pi.nombre LIKE '%Iguazu%'
  AND hp.distancia_km <= 15.00
  AND r.estado = 'confirmada'
GROUP BY pi.id_punto, pi.nombre, h.id_hotel, h.nombre, hp.distancia_km
ORDER BY recaudacion_total DESC;
```

**Consulta Q05: Recaudación por Categoría de Habitación en Ciudad Turística (4 tablas)**

```sql
SELECT
    h.ciudad,
    th.nombre AS categoria,
    COUNT(r.id_reserva) AS cantidad_reservas,
    ROUND(AVG(r.tarifa), 2) AS tarifa_media,
    SUM(r.tarifa) AS ingresos_totales
FROM HOTEL h
JOIN HABITACION hab ON h.id_hotel = hab.id_hotel
JOIN TIPO_HABITACION th ON hab.id_tipo_hab = th.id_tipo_hab
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE h.ciudad = 'San Carlos de Bariloche'
  AND r.estado = 'confirmada'
GROUP BY h.ciudad, th.id_tipo_hab, th.nombre
ORDER BY ingresos_totales DESC;
```

**Consulta Q06: Disponibilidad de Habitación mediante NOT EXISTS (Punto 26)**

```sql
SELECT
    h.nombre AS hotel,
    hab.numero_habitacion,
    th.nombre AS tipo
FROM HABITACION hab
JOIN HOTEL h ON hab.id_hotel = h.id_hotel
JOIN TIPO_HABITACION th ON hab.id_tipo_hab = th.id_tipo_hab
WHERE h.ciudad = 'San Carlos de Bariloche'
  AND NOT EXISTS (
      SELECT 1
      FROM RESERVA r
      WHERE r.id_habitacion = hab.id_habitacion
        AND r.estado IN ('confirmada', 'pendiente')
        AND r.fecha_inicio < '2024-07-20'
        AND r.fecha_salida > '2024-07-10'
  )
LIMIT 20;
```

**Consulta Q07: Disponibilidad Alternativa con LEFT JOIN Excluyente (Punto 26)**

```sql
SELECT
    hab.id_habitacion,
    hab.numero_habitacion,
    h.nombre AS hotel
FROM HABITACION hab
JOIN HOTEL h ON hab.id_hotel = h.id_hotel
```

```
LEFT JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
    AND r.estado IN ('confirmada', 'pendiente')
    AND r.fecha_inicio < '2024-07-20'
    AND r.fecha_salida > '2024-07-10'
WHERE h.ciudad = 'San Carlos de Bariloche'
  AND r.id_reserva IS NULL
LIMIT 20;
```

**Consulta Q08: Top 5 Hoteles con Mayor Recaudación Anual (Punto 27)**

```
SELECT
    h.nombre AS hotel,
    COUNT(r.id_reserva) AS total_reservas,
    SUM(r.tarifa) AS ingresos_totales
FROM HOTEL h
JOIN HABITACION hab ON h.id_hotel = hab.id_hotel
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE r.estado = 'confirmada'
  AND r.fecha_inicio >= '2023-01-01'
  AND r.fecha_inicio <= '2023-12-31'
GROUP BY h.id_hotel, h.nombre
ORDER BY ingresos_totales DESC
LIMIT 5;
```

**Consulta Q09: Evolución Mensual y Tasa de Cancelación con CASE WHEN (Punto 27)**

```
SELECT
    DATE_FORMAT(r.fecha_inicio, '%Y-%m') AS periodo,
    COUNT(r.id_reserva) AS total_operaciones,
    SUM(CASE WHEN r.estado = 'confirmada' THEN 1 ELSE 0 END) AS confirmadas,
    SUM(CASE WHEN r.estado = 'cancelada' THEN 1 ELSE 0 END) AS canceladas,
    SUM(CASE WHEN r.estado = 'pendiente' THEN 1 ELSE 0 END) AS pendientes,
    ROUND(SUM(CASE WHEN r.estado = 'cancelada' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS
tasa_cancelacion_pct
FROM RESERVA r
WHERE r.fecha_inicio BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY periodo
ORDER BY periodo ASC;
```

**Consulta Q10: Duración Promedio de Estadías por Ciudad (DATEDIFF) (Punto 27)**

```
SELECT
    h.ciudad,
    YEAR(r.fecha_inicio) AS anio,
    ROUND(AVG(DATEDIFF(r.fecha_salida, r.fecha_inicio)), 1) AS promedio_noches,
    MAX(DATEDIFF(r.fecha_salida, r.fecha_inicio)) AS maxima_estadia
FROM HOTEL h
JOIN HABITACION hab ON h.id_hotel = hab.id_hotel
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE r.estado = 'confirmada'
GROUP BY h.ciudad, anio
HAVING anio >= 2022
ORDER BY h.ciudad, anio;
```

**Consulta Q11: Huéspedes Frecuentes en 3+ Hoteles Distintos (Punto 24)**

```
SELECT
    hue.id_huesped,
    hue.apellido,
    hue.nombre,
```

```
    COUNT(DISTINCT hab.id_hotel) AS hoteles_distintos_visitados,
    COUNT(r.id_reserva) AS total_estadias
FROM HUESPED hue
JOIN RESERVA r ON hue.id_huesped = r.id_huesped
JOIN HABITACION hab ON r.id_habitacion = hab.id_habitacion
WHERE r.estado = 'confirmada'
GROUP BY hue.id_huesped, hue.apellido, hue.nombre
HAVING COUNT(DISTINCT hab.id_hotel) >= 3
ORDER BY hoteles_distintos_visitados DESC, total_estadias DESC
LIMIT 25;
```

**Consulta Q12: Habitaciones de Alta Ocupación sin Cancelaciones (Punto 24)**

```
SELECT
    hab.id_habitacion,
    h.nombre AS hotel,
    hab.numero_habitacion,
    COUNT(r.id_reserva) AS total_reservas_exitosas
FROM HABITACION hab
JOIN HOTEL h ON hab.id_hotel = h.id_hotel
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE r.estado = 'confirmada'
  AND NOT EXISTS (
      SELECT 1
      FROM RESERVA sub_r
      WHERE sub_r.id_habitacion = hab.id_habitacion
        AND sub_r.estado = 'cancelada'
  )
GROUP BY hab.id_habitacion, h.nombre, hab.numero_habitacion
HAVING total_reservas_exitosas > 500
ORDER BY total_reservas_exitosas DESC
LIMIT 15;
```

**Consulta Q13: Hoteles cuya Tarifa Media Supera la Media Global (Punto 24)**

```
SELECT
    h.id_hotel,
    h.nombre AS hotel,
    ROUND(AVG(r.tarifa), 2) AS tarifa_promedio_hotel
FROM HOTEL h
JOIN HABITACION hab ON h.id_hotel = hab.id_hotel
JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE r.estado = 'confirmada'
GROUP BY h.id_hotel, h.nombre
HAVING AVG(r.tarifa) > (
    SELECT AVG(tarifa)
    FROM RESERVA
    WHERE estado = 'confirmada'
)
ORDER BY tarifa_promedio_hotel DESC;
```

**Consulta Q14: Impacto del Transporte de Datos (SELECT * vs Proyección) (Punto 28)**

```
-- Versión A: Transporte masivo de columnas
SELECT * FROM RESERVA WHERE estado = 'confirmada' AND fecha_inicio BETWEEN '2023-01-01' AND '2023-03-31' LIMIT
10000;

-- Versión B: Proyección selectiva de atributos esenciales
SELECT id_reserva, id_habitacion, fecha_inicio, fecha_salida, tarifa
FROM RESERVA
WHERE estado = 'confirmada' AND fecha_inicio BETWEEN '2023-01-01' AND '2023-03-31' LIMIT 10000;
```

**Consulta Q15: Paginación Profunda (OFFSET vs Keyset Pagination) (Punto 28)**

```
-- Versión A (Ingenua con OFFSET): Escanea y descarta un millón de registros
SELECT id_reserva, id_huesped, fecha_inicio, tarifa FROM RESERVA WHERE estado = 'confirmada' ORDER BY
id_reserva LIMIT 20 OFFSET 1000000;

-- Versión B (Keyset / Seek Pagination): Acceso directo indexado
SELECT id_reserva, id_huesped, fecha_inicio, tarifa FROM RESERVA WHERE estado = 'confirmada' AND id_reserva >
1000000 ORDER BY id_reserva LIMIT 20;
```

# PARTE VI — OPTIMIZADOR, SARGABILIDAD Y PLANES DE EJECUCIÓN

### Punto 29: Diagnóstico con EXPLAIN y EXPLAIN ANALYZE

- **Concepto:** `EXPLAIN` presenta la estimación estática del Cost-Based Optimizer (CBO). `EXPLAIN ANALYZE` ejecuta efectivamente la consulta instrumentando el tiempo real de CPU por nodo del árbol, la cantidad exacta de filas procesadas y las iteraciones de bucle (*loops*).

### Punto 30: SARGabilidad y Predicados Optimizables

- **Definición:** Un predicado es SARGable (*Search Argument Able*) cuando el motor puede aislar la columna indexada en un extremo del operador relacional sin transformarla mediante funciones.
- **Evidencia Empírica de EXPLAIN:**
  - No-SARGable (`WHERE YEAR(fecha_inicio) = 2023`): `type: index`, procesando **7.794.965 filas**.
  - SARGable (`WHERE fecha_inicio >= '2023-01-01' AND fecha_inicio < '2024-01-01'`): `type: range`, `key_len: 3`, navegando el árbol en **0,42 segundos**.

### Puntos 31 al 33: Costos, Estadísticas y Optimización de JOINs

- **Algoritmo de Emparejamiento:** MySQL utiliza *Nested Loop Join*. Sin índices foráneos, por cada habitación examinada, el motor ejecutaba un escaneo secuencial de los 8.1M de reservas (complejidad O(M × N)).
- **Solución:** La creación de los índices `idx_reserva_habitacion` y `idx_hab_hotel` redujo la complejidad algorítmica a O(M × log N), transformando el acceso de tipo `ALL` a `ref`.

# PARTE VII — OPTIMIZACIÓN DE CONSULTAS Y TABLA COMPARATIVA

### Puntos 34 al 37: La Gran Tabla Comparativa "Antes vs. Después" de las 8 Consultas Críticas

Mediciones reales obtenidas mediante `EXPLAIN ANALYZE` en el contenedor de base de datos con **8.100.000 de registros**:

| Consulta | Descripción / Problema Inicial | Plan de Ejecución Inicial (Sin Índices) | Tiempo Antes | Estrategia de Optimización / Índices Creados | Plan de Ejecución Optimizado | Tiempo Después | % de Mejora |
|---|---|---|---|---|---|---|---|
| **Q08** | Ranking Top 5 hoteles por recaudación anual | `Table scan on RESERVA` (8.1M filas examinadas) | **67,64 s** | `idx_hab_hotel` + `idx_reserva_disponibilidad` | Index lookup en claves foráneas + Condition pushdown | **6,84 s** | **89,9%** |
| **Q06** | Disponibilidad con `NOT EXISTS` en Bariloche | Table scan por cada habitación candidata | **62,10 s** | `idx_reserva_disponibilidad(id_hab, estado, inicio, fin)` | Covering index lookup en Antijoin | **0,31 s** | **99,5%** |

| Consulta | Descripción / Problema Inicial | Plan de Ejecución Inicial (Sin Índices) | Tiempo Antes | Estrategia de Optimización / Índices Creados | Plan de Ejecución Optimizado | Tiempo Después | % de Mejora |
|---|---|---|---|---|---|---|---|
| **Q01** | Detalle de reservas por hotel y huésped (4 tablas) | Full Table Scan en `RESERVA` | **48,20 s** | Claves foráneas indexadas `idx_res_hab` e `idx_res_huesped` | Nested Loop Join por `ref` | **0,85 s** | **98,2%** |
| **Q03** | Top 10 huéspedes con mayor gasto histórico | Full scan + Sort en disco | **54,30 s** | `idx_reserva_huesped_tarifa(id_huesped, estado, tarifa)` | Covering index aggregation | **1,20 s** | **97,8%** |
| **Q09** | Evolución mensual por estados (`CASE WHEN`) | Table scan completo | **26,40 s** | `idx_reserva_fecha_estado(fecha_inicio, estado)` | Covering index range scan | **0,78 s** | **97,0%** |
| **Q11** | Clientes frecuentes en 3+ hoteles (`HAVING`) | Full scan + Temporary table | **41,50 s** | `idx_reserva_huesped` | Direct index lookup por cliente | **1,95 s** | **95,3%** |
| **Q14** | Transporte masivo: `SELECT *` vs selectivo | Lectura de fila completa en disco | **12,40 s** | Proyección acotada de atributos estrictos | Menor serialización y carga de red | **1,80 s** | **85,5%** |
| **Q15** | Paginación profunda (1.000.000 filas de salto) | `LIMIT 20 OFFSET 1000000` (escanea 5.6M filas) | **52,67 s** | Keyset Pagination (`WHERE id_reserva > 1000000`) | Direct index range scan por `PRIMARY` | **0,0002 s (0.2 ms)** | **99,9996%** |

# PARTE VIII — TRANSACCIONES Y CONCURRENCIA

### Punto 38: Mecanismos de Bloqueo a Nivel de Tabla y Fila (S y X)

- **Bloqueo Compartido (*Shared - S*):** `SELECT ... FOR SHARE`. Permite lecturas concurrentes, pero impide que cualquier otra transacción modifique la fila.
- **Bloqueo Exclusivo (*Exclusive - X*):** `SELECT ... FOR UPDATE` o sentencias `UPDATE`/`DELETE`. Otorga posesión exclusiva para modificación.

---

### Punto 39: Concurrencia entre Dos o Más Conexiones Simultáneas

Cuando la Conexión 1 ejecuta `UPDATE RESERVA SET tarifa = 100 WHERE id_reserva = 1;` dentro de una transacción sin confirmar (`START TRANSACTION`), la Conexión 2 intenta modificar la misma tupla y queda suspendida en estado `LOCK WAIT`. Si la Conexión 1 no hace `COMMIT` antes de los 50 segundos (`innodb_lock_wait_timeout`), MySQL aborta la Conexión 2 con:

```
ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
```

---

### Punto 40: Niveles de Aislamiento ANSI SQL y Fenómenos Anómalos

- **Fenómenos Clásicos:**
  - *Lectura Sucia (Dirty Read):* Leer datos no confirmados por otra transacción que luego hace `ROLLBACK`.
  - *Lectura No Repetible (Non-repeatable Read):* Volver a leer la misma fila y encontrar valores modificados por otra transacción confirmada.

    - *Lectura Fantasma (Phantom Read):* Una consulta por rango encuentra nuevas filas insertadas por otra transacción confirmada.
- **Comportamiento en InnoDB (`REPEATABLE READ`):** InnoDB utiliza **MVCC** con vistas consistentes en memoria para evitar lecturas sucias y no repetibles. Adicionalmente, utiliza **Next-Key Locks** (bloqueo del registro y del intervalo vacío *Gap Lock* previo) en búsquedas indexadas, eliminando por completo las lecturas fantasmas sin degradar la concurrencia a nivel `SERIALIZABLE`.

---

Punto 41: Simulación Experimental de Deadlocks en 4 Pasos

- **Guía de Ejecución en Dos Terminales:**

```
================== CONEXIÓN 1 ==================
PASO 1:
START TRANSACTION;
UPDATE RESERVA SET tarifa = 111.00 WHERE id_reserva = 1;

================== CONEXIÓN 2 ==================
PASO 2:
START TRANSACTION;
UPDATE RESERVA SET tarifa = 222.00 WHERE id_reserva = 2;

================== CONEXIÓN 1 ==================
PASO 3:
UPDATE RESERVA SET tarifa = 111.00 WHERE id_reserva = 2;
--> (Queda bloqueada esperando que Conexión 2 libere la fila 2)

================== CONEXIÓN 2 ==================
PASO 4:
UPDATE RESERVA SET tarifa = 222.00 WHERE id_reserva = 1;
--> ¡¡DEADLOCK DETECTADO DE FORMA INMEDIATA POR INNODB!!
```

- **Salida Oficial del Motor Capturada en Conexión 2:**

```
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

- **Resolución Automática:** El detector `innodb_deadlock_detect = ON` identifica el ciclo cerrado en el grafo de dependencias y selecciona como transacción víctima a aquella que menor cantidad de registros modificó en su Undo Log, aplicándole un `ROLLBACK` forzado para liberar a la otra conexión.

---

Punto 42: Herramientas Forenses y Diagnóstico de Bloqueos en InnoDB

- **Inspección en Vivo:**

```
SELECT * FROM performance_schema.data_locks;
SELECT * FROM performance_schema.data_lock_waits;
```

- **Análisis Forense Post-Mortem:** La sección `LATEST DETECTED DEADLOCK` de `SHOW ENGINE INNODB STATUS\G` detalla el timestamp exacto, los hilos involucrados, las sentencias SQL colisionadas y las páginas de disco sobre las cuales se produjo el bloqueo mutuo.

---

# PARTE IX — MEMORIA, SERVIDOR Y RENDIMIENTO

Punto 43: Configuración del Motor y Archivo my.cnf

Parámetros clave aplicados en `config/my.cnf`:

- `innodb_buffer_pool_size = 268435456` (256 MB): Tamaño controlado para evaluar el intercambio de páginas LRU.
- `disable_log_bin = 1`: Desactiva el registro binario de replicación para evitar I/O redundante durante la carga masiva.
- `local_infile = 1`: Habilitación de ingesta masiva acelerada.

---

### Punto 44: Telemetría Real del InnoDB Buffer Pool (Hit Ratio del 86,11%)

Valores reales medidos en el servidor tras la ejecución del benchmark:

- **Páginas Totales:** `16.384 páginas` de 16 KB = **256 MB**.
- **Lecturas Lógicas Servidas en RAM:** `32.004.858 lecturas` (*Innodb_buffer_pool_read_requests*).
- **Lecturas Físicas desde Disco SSD:** `4.444.634 lecturas` (*Innodb_buffer_pool_reads*).
- **Tasa de Aciertos de Caché (Hit Ratio):**

> **Hit Ratio = (32.004.858 - 4.444.634) / 32.004.858 = 86,11%**
> *Diagnóstico:* El 86,11% de las peticiones se sirvieron en microsegundos directamente desde la memoria RAM.

---

### Punto 45: Identificación de Cuellos de Botella: CPU, Disco y Red

- **I/O-Bound (Almacenamiento):** Consultas en Cold Cache con cola de lectura en disco saturada.
- **CPU-Bound (Procesamiento):** Agregaciones masivas (`SUM`, `AVG`) y ordenamientos en memoria sobre páginas residentes en el Buffer Pool.
- **Network-Bound (Ancho de Banda):** Consultas `SELECT *` masivas de 8 millones de tuplas que saturan el socket TCP de red por transferencia de datos no proyectados.

---

### Punto 46: Evaluación de Pools de Conexión (Connection Pooling)

Abrir una conexión directa implica un costo repetitivo de tres vías: negociación TCP, autenticación con cifrado TLS y asignación de memoria privada por hilo (`sort_buffer_size`). Un Connection Pool (como HikariCP) mantiene conexiones calientes preasignadas, reduciendo la latencia de conexión de **20 ms a 0,1 ms**.

---

### Punto 47: Configuración y Evidencia del Slow Query Log

- **Parámetros Configurados:**

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1.0;
SET GLOBAL log_queries_not_using_indexes = 'ON';
```

- **Evidencia Real Capturada en `/var/lib/mysql/...-slow.log`:**

```
# Time: 2026-10-04T05:33:16.220922Z
# User@Host: admin[admin] @ localhost []
# Query_time: 67.645186  Lock_time: 0.000037  Rows_sent: 5  Rows_examined: 6646105
SELECT h.nombre AS hotel, COUNT(r.id_reserva) AS total_reservas, SUM(r.tarifa) AS ingresos_totales
FROM HOTEL h JOIN HABITACION hab ON h.id_hotel = hab.id_hotel JOIN RESERVA r ON hab.id_habitacion =
r.id_habitacion
WHERE r.estado = 'confirmada' AND r.fecha_inicio BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY h.id_hotel, h.nombre ORDER BY ingresos_totales DESC LIMIT 5;
```

---

### Punto 48: Tareas Preventivas de Mantenimiento de Tablas e Índices

- `ANALYZE TABLE RESERVA;`: Recalcula el muestreo estadístico de cardinalidad en el B+Tree para que el optimizador no seleccione índices subóptimos.
- `OPTIMIZE TABLE RESERVA;`: Reconstruye físicamente el tablespace `.ibd`, desfragmentando páginas de 16 KB vaciadas por operaciones de borrado.

---

# PARTE X — ESCALABILIDAD, PARTICIONAMIENTO Y ESTRUCTURAS AVANZADAS

---

### Punto 49: Análisis Comparativo: Escalabilidad Vertical vs. Horizontal

- **Escalabilidad Vertical (*Scale-Up*):** Incrementar CPU, memoria RAM y almacenamiento NVMe en un único servidor. Preserva la coherencia ACID pura y la simplicidad operacional, pero encuentra un límite físico y económico de hardware.
- **Escalabilidad Horizontal (*Scale-Out*):** Distribuir la base en múltiples nodos (*Sharding* o *Read Replicas*). Supera los límites físicos de una máquina pero introduce la complejidad del Teorema CAP y la latencia del protocolo Two-Phase Commit (2PC) para transacciones distribuidas.

---

Punto 50: Implementación Física de Particionamiento Horizontal por Rangos

- **Restricción Arquitectónica de InnoDB:** La columna de particionamiento debe formar parte obligatoria de la clave primaria: `PRIMARY KEY (id_reserva, fecha_inicio)`.
- **Script DDL Implementado (`scripts/08_particionamiento_y_vistas.sql`):**

```
DROP TABLE IF EXISTS RESERVA_PARTICIONADA;

CREATE TABLE RESERVA_PARTICIONADA (
    id_reserva INT UNSIGNED NOT NULL,
    id_huesped INT UNSIGNED NOT NULL,
    id_habitacion INT UNSIGNED NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_salida DATE NOT NULL,
    tarifa DECIMAL(10,2) NULL,
    estado VARCHAR(20) NOT NULL,
    PRIMARY KEY (id_reserva, fecha_inicio)
) ENGINE=InnoDB
PARTITION BY RANGE (YEAR(fecha_inicio)) (
    PARTITION p_antiguas VALUES LESS THAN (2020),
    PARTITION p2020 VALUES LESS THAN (2021),
    PARTITION p2021 VALUES LESS THAN (2022),
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p_futuras VALUES LESS THAN MAXVALUE
);
```

---

Punto 51: Demostración Empírica de Partition Pruning en Planes de Ejecución

- **Consulta con Partition Pruning (Filtro por Año 2023):**

```
EXPLAIN SELECT * FROM RESERVA_PARTICIONADA WHERE fecha_inicio BETWEEN '2023-01-01' AND '2023-12-31';
```

*Salida de EXPLAIN:*

```
partitions: p2023 | type: ALL | rows: 62.933 | Extra: Using where
```

*Diagnóstico:* El motor **accede exclusivamente al archivo físico de la partición** `p2023`, podando y descartando por completo las 7 particiones restantes.

- **Consulta sin Partition Pruning (Filtro por Columna No Particionada):**

```
EXPLAIN SELECT * FROM RESERVA_PARTICIONADA WHERE estado = 'confirmada';
```

*Salida de EXPLAIN:*

```
partitions: p_antiguas,p2020,p2021,p2022,p2023,p2024,p2025,p_futuras | rows: 500.041
```

*Diagnóstico:* Al no filtrar por la clave de partición, el motor debe escanear todas las particiones del disco.

Punto 52: Vistas Estándar versus Vistas Materializadas

- **Vistas Dinámicas Estándar:** Objetos virtuales que re-ejecutan el árbol de consultas en cada lectura.

```
CREATE OR REPLACE VIEW v_recaudacion_mensual_dinamica AS
SELECT hab.id_hotel, DATE_FORMAT(r.fecha_inicio, '%Y-%m') AS anio_mes,
       COUNT(r.id_reserva) AS cantidad_reservas, SUM(r.tarifa) AS total_recaudado
FROM HABITACION hab JOIN RESERVA r ON hab.id_habitacion = r.id_habitacion
WHERE r.estado = 'confirmada' GROUP BY hab.id_hotel, anio_mes;
```

- **Vistas Materializadas / Tablas de Resumen:** Persistencia física de los resultados en `REPORTE_RECAUDACION_MENSUAL` mantenida por triggers.
- **Comparativa de Costo en EXPLAIN:**
  - Vista Dinámica: Ejecuta un *Derived Table Scan* con agregación en memoria procesando cientos de tuplas.
  - Tabla Materializada: Búsqueda indexada directa en tiempo **O(1)** por clave primaria `(id_hotel, anio_mes)`, reduciendo el consumo de CPU a cero.

# CONCLUSIONES FINALES

1. **La Optimización es una Disciplina de Compromisos (*Trade-Offs*):** No existe la "base de datos perfecta". Diseñar índices para acelerar lecturas penaliza las operaciones de escritura. El rol del ingeniero de bases de datos consiste en calibrar los árboles B+Tree en función de la criticidad de la carga de trabajo.
2. **El Dominio de la Arquitectura Interna de InnoDB:** Comprender el funcionamiento del Buffer Pool, las páginas de 16 KB, el índice agrupado y la regla del prefijo izquierdo permitió reducir consultas de **más de 1 minuto a fracciones de milisegundo**, demostrando que la optimización de código y diseño supera con creces el simple incremento de hardware.
3. **Reproducibilidad:** El empleo de Docker Compose y scripts SQL reproducibles garantiza que la totalidad de los experimentos presentados en este informe son verificables y defendibles ante la cátedra en cualquier entorno informático.